

AI-Based System Intent Engine

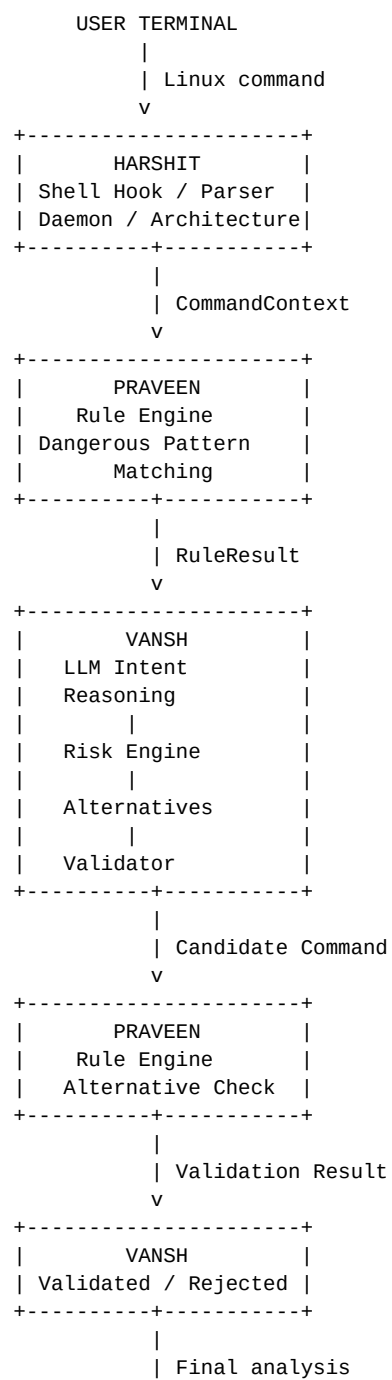
Complete 3-Member Pipeline & Data-Flow Contract

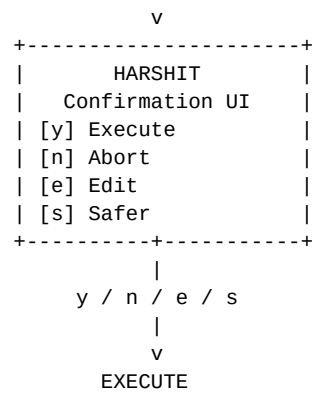
Team ownership: Harshit — Shell Hook, Daemon, Parser & Architecture · Praveen — Deterministic Rule Engine · Vansh — LLM Reasoning, Risk Refinement & Safer Alternatives

Purpose

This document defines exactly how the three team members' modules communicate, what each member receives, what each member produces, and where the integration boundaries exist. The goal is to keep the architecture modular so each member can develop and test independently before final integration.

Complete 3-Member Pipeline





Core principle: Harshit handles terminal interception and execution, Praveen performs deterministic rule-based analysis, and Vansh performs semantic understanding, risk refinement, safer-alternative generation, and alternative validation.

1. Harshit → Praveen

Harshit owns the shell interception and parser.

User types:

```
rm -rf ./project
```

Harshit captures the command before execution and sends a command context to Praveen.

Example input:

```
{ "command": "rm -rf ./project", "shell": "bash", "cwd": "/home/vansh/project",  
  "timestamp": "2026-08-19T10:30:00", "parsed": { "commands": [ { "program": "rm",  
    "arguments": ["-rf", "./project"] } ] } }
```

The essential information is the command, shell/context, and parsed command structure. Not every field is mandatory on Day 1.

2. Praveen's Output → Vansh

This is the first integration contract. Praveen's Rule Engine analyzes the command and returns a RuleResult.

Example command:

```
find . -type f -delete
```

```
{ "command": "find . -type f -delete", "classification": "AMBIGUOUS",  
  "confidence": 0.42, "matched_rules": [], "context": { "shell": "bash", "cwd":  
    "/home/vansh/project" } }
```

This RuleResult is the primary input to Vansh's module. Vansh does not need to receive raw shell input directly in the final architecture.

3. Vansh — Intent Reasoner

Vansh takes the RuleResult and supplies relevant information to the local LLM (Ollama).

LLM reasoning context:

COMMAND: find . -type f -delete

RULE ENGINE CLASSIFICATION: AMBIGUOUS

RULE ENGINE CONFIDENCE: 0.42

SHELL: bash

CURRENT WORKING DIRECTORY: /home/vansh/project

Example LLM response:

```
{ "risk_level": "MEDIUM", "confidence": 0.42, "intent": "Delete all regular files  
in the current directory and subdirectories", "impact": "Potential data loss",  
  "affected_resources": ["filesystem"], "reversible": false,  
  "requires_confirmation": true, "explanation": "..." }
```

This is the LLMIntentResponse. The Reasoner explicitly transforms it into the application's IntentResult after validation and sanity checks.

4. Vansh — Risk Engine

The Risk Engine takes the IntentResult and applies deterministic safety policy. This layer remains inside Vansh's module.

```
Input: { "risk_level": "MEDIUM", "confidence": 0.42, "intent": "Delete all
regular files...", "impact": "Potential data loss", "reversible": false } Output:
{ "risk_level": "MEDIUM", "confidence": 0.42, "requires_confirmation": true }
```

The purpose is to avoid relying blindly on the LLM for safety-critical decisions.

5. Vansh — Alternative Generator

The Alternative Generator converts the semantic intent into a safer strategy and candidate command.

```
Input: { "command": "find . -type f -delete", "intent": "Delete all regular
files", "risk_level": "MEDIUM", "reversible": false } Output: {
"original_command": "find . -type f -delete", "candidate_command": "find . -type
f -print", "intent": "Delete all regular files", "strategy": "Inspect before
deletion", "explanation": "Lists matching files without deleting them.",
"status": "GENERATED" }
```

This output is represented by AlternativeResult.

6. Vansh → Praveen — Alternative Validation

This is the second integration between the two modules. Vansh's validator sends the candidate command back to Praveen's Rule Engine.

```
{ "command": "find . -type f -print", "shell": "bash", "cwd":
"/home/vansh/project" }
```

Praveen analyzes the candidate and can return:

```
{ "command": "find . -type f -print", "classification": "SAFE", "confidence":
0.98, "matched_rules": [], "context": { "shell": "bash", "cwd":
"/home/vansh/project" } }
```

Vansh then converts SAFE → VALIDATED. If the candidate is AMBIGUOUS or otherwise not confirmed safe, it is rejected rather than presented as safe.

This is a request-response loop: YOU → candidate command → PRAVEEN → RuleResult → YOU.

7. Vansh → Harshit

After semantic analysis and alternative validation, Vansh provides the daemon/UI layer with enough information to present the decision to the user.

```
{ "original_command": "find . -type f -delete", "risk_level": "MEDIUM", "intent":
"Delete all regular files", "impact": "Potential data loss",
"requires_confirmation": true, "explanation": "This permanently deletes matching
files.", "alternative": { "command": "find . -type f -print", "strategy":
"Inspect before deletion", "status": "VALIDATED" } }
```

Harshit's confirmation UI can display:

■ MEDIUM RISK

Intent: Delete all regular files.

Impact: Potential data loss.

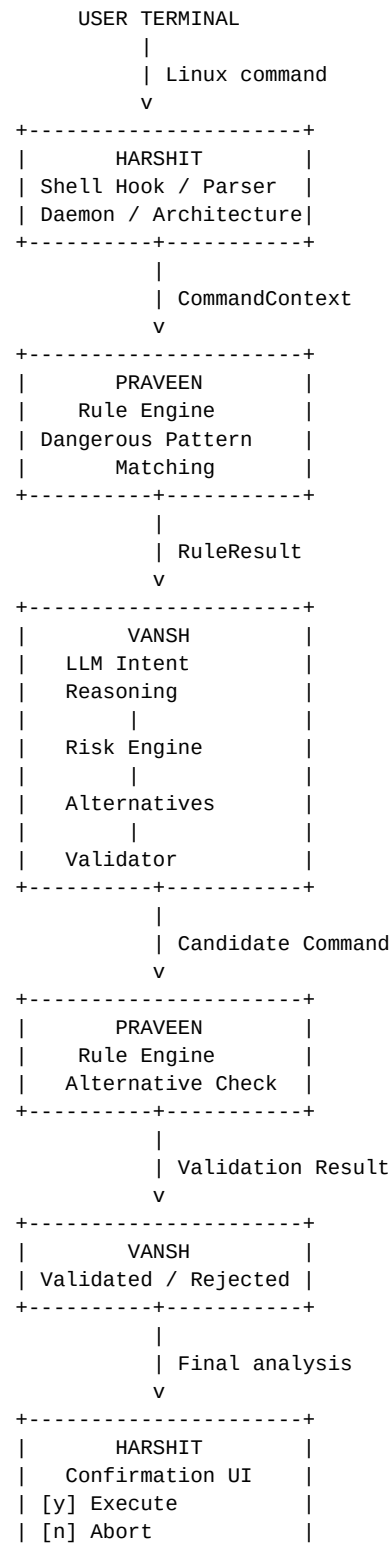
Safer alternative: `find . -type f -print`

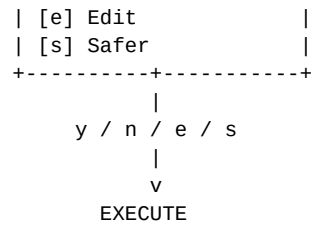
[y] Execute [n] Abort [e] Edit [s] Use safer

Three Members' Boundaries

Member	Input	Responsibilities	Output
Harshit	User terminal / system events	Shell interception, parser, daemon, routing, confirmation UI, Confirmation	ExecutionContext → Praveen; FinalDecision → terminal
Praveen	CommandContext; later candidate alternatives	Declarative Rule Engine and dangerous-pattern matching	RuleResult → Vansh; RuleResult → Vansh for alternatives
Vansh	Primary input: RuleResult	LLM reasoning, IntentResult transformation, risk refinement, Safe analysis, Validator	Final analysis, Validated Result; candidate commands

Most Important Integration Diagram





In One Sentence

Harshit handles the terminal and execution, Praveen determines known rule-based risk, and Vansh handles semantic understanding, risk refinement, and safer alternatives, with Praveen re-validating every generated alternative before it can be considered safe.

Architecture boundary: Vansh should not receive the raw command directly from Harshit in the final architecture. His primary contract is RuleResult from Praveen. This separation keeps the three-member architecture clean and independently testable.